COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10/13/2026	**DATE SUBMITTED:**	10/ /2026
NAME:	Ralf Nicolo B. Abello Rodney C. Domingo	**INSTRUCTOR:**	Engr. Jimlord Quejado

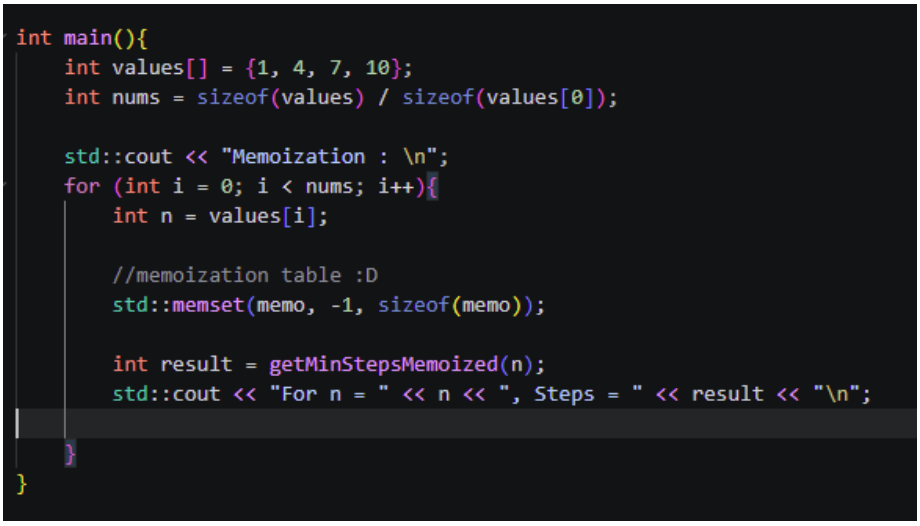
ACTIVITY NO. 12 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Linked List	The function calls itself on smaller parts of the array. First, **partition()** places one number (the pivot) in its correct spot at index **p**. Then **quickSort(arr, low, p - 1)** sorts the left side and **quickSort(arr, p + 1, high)** sorts the right side. Each call gets fewer elements than the one before it. The base case is **low >= high**, which means the part has 0 or 1 elements. It is already sorted, so the function stops.
Brute Force	Linear Search	The loop checks the array one element at a time, starting at **arr[0]**, and compares each one to the **key**. It stops when **arr[i] == key** or when the array ends. There are no shortcuts. It simply tries every element until one matches, like trying every key on a keyring until one opens the lock or guessing your forgotten password
Backtracking	Depth-First Search (DFS)	The search starts at one vertex, marks it **visited**, and moves to an unvisited neighbor, going as deep as possible. If a vertex has no unvisited neighbors, it is a dead end. The function then returns to the previous vertex and tries

		another neighbor. Going forward, hitting a dead end, stepping back, and trying another path is backtracking. explored one branch at a time
Greedy	Selection Sort	On each pass, the algorithm finds the smallest number in the unsorted part (**min_idx**) and swaps it into position **i**. It always picks what looks best at that moment and never changes a choice it already made. After each pass, the sorted part grows by one element.
Divide-and-Conquer	Merge Sort	The array is split in half at **mid** (divide). The left half and right half are sorted separately (solve). Then **merge()** joins the two sorted halves into one sorted array (combine). These three steps match divide, solve, and combine.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	

```cpp
int main(){
    int values[] = {1, 4, 7, 10};
    int nums = sizeof(values) / sizeof(values[0]);

    std::cout << "Memoization : \n";
    for (int i = 0; i < nums; i++){
        int n = values[i];

        //memoization table :D
        std::memset(memo, -1, sizeof(memo));

        int result = getMinStepsMemoized(n);
        std::cout << "For n = " << n << ", Steps = " << result << "\n";

    }
}
```

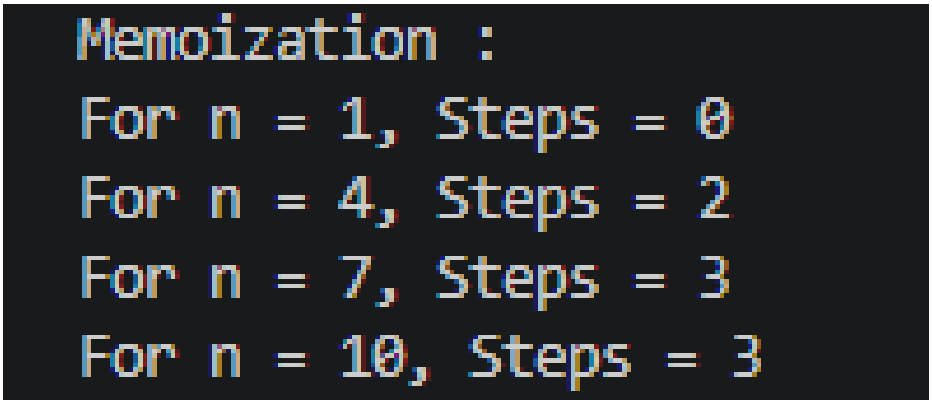
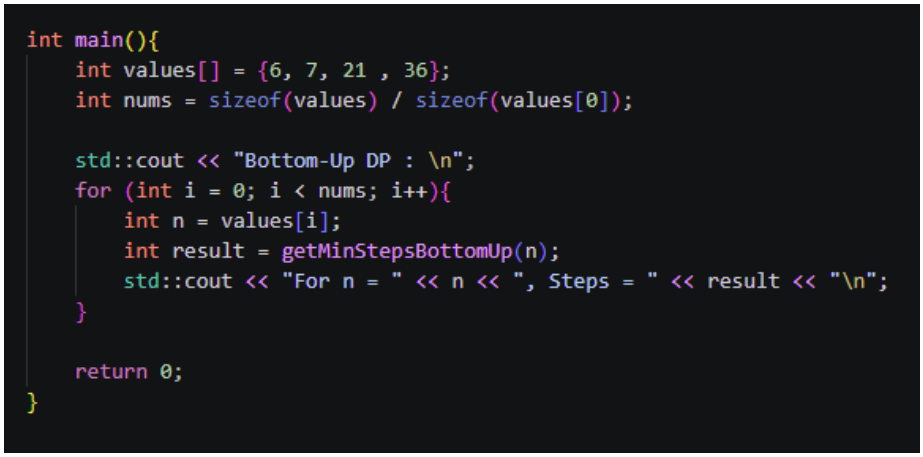
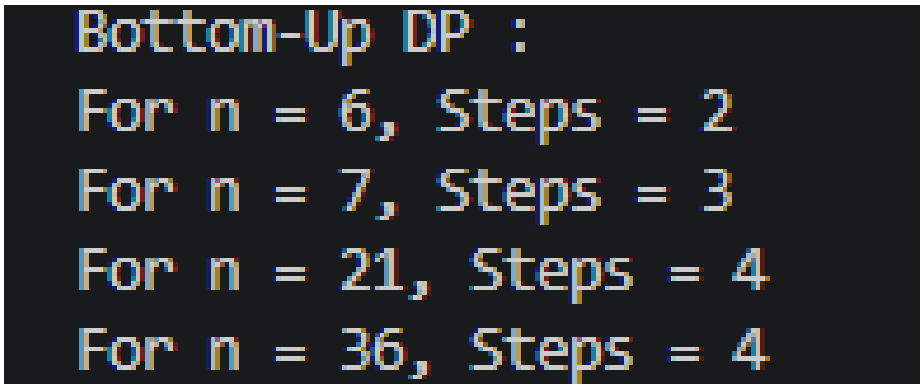
	
Analysis	This main driver file implements a top-down memoization approach by recursively breaking down the problem for each value, using a memoization table, it initialized with the “std::memset” function to store and reuse solved subproblems, therefore eliminating redundant recursive calls and efficiently computing the steps to reach one.

Table 12-2. Memoization Implementation

Screenshot	 
Analysis	While this main driver file implements the bottom-up dynamic programming approach

	by iteratively computing and storing solutions from trivial base cases up to each values, utilizing a DP table to systematically determine the steps required to reach one without relying on recursive function calls.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode (+ Solving Problem by Hand)

```
FUNCTION countPaths(i, j, cost)
   IF i < 0 OR j < 0 OR cost < 0 THEN
      RETURN 0
   IF i == 0 AND j == 0 THEN
      IF cost == matrix[0][0] THEN RETURN 1
      ELSE RETURN 0
   remaining = cost - matrix[i][j]
   RETURN countPaths(i-1, j, remaining) + countPaths(i, j-1, remaining)
END FUNCTION

MAIN
   PRINT countPaths(M-1, N-1, target)
```

CODE (Abello)

```cpp
#include <iostream>

int countPaths(int matrix[][4], int row, int col, int cost)
{
    if (cost < 0)
    {
        return 0;
    }

    if (row == 0 && col == 0)
    {
        return (cost == matrix[0][0]) ? 1 : 0;
    }

    if (row == 0)
    {
        return countPaths(
            matrix,
            0,
            col - 1,
            cost - matrix[row][col]
        );
    }

    if (col == 0)
    {
        return countPaths(
            matrix,
            row - 1,
            0,
            cost - matrix[row][col]
        );
    }

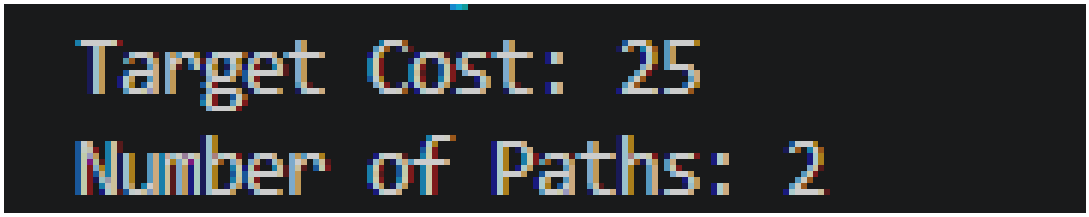
```

```
34
35      return countPaths(
36          matrix,
37          row - 1,
38          col,
39          cost - matrix[row][col]
40      )
41      +
42      countPaths(
43          matrix,
44          row,
45          col - 1,
46          cost - matrix[row][col]
47      );
48  }
49
50  int main()
51  {
52      int matrix[4][4] =
53      {
54          {4, 7, 1, 6},
55          {5, 7, 3, 9},
56          {3, 2, 1, 2},
57          {7, 1, 6, 3}
58      };
59
60      int cost = 25;
61
62      std::cout << "Target Cost: "
63                << cost << std::endl;
64
65      std::cout << "Number of Paths: "
66                << countPaths(matrix, 3, 3, cost)
67                << std::endl;
68
69      return 0;
70  }
```

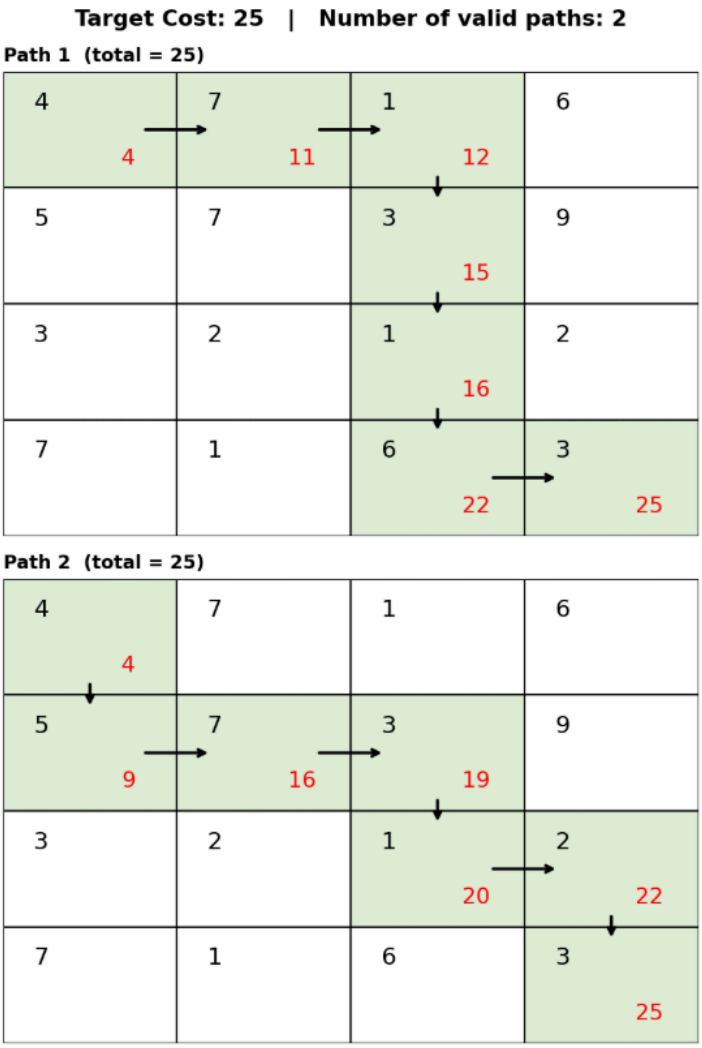
Analysis of the Working Code (Abello)
I ran **countPaths(matrix, 3, 3, 25)**, which starts at the last cell and works backward toward (0, 0). At each cell, I subtracted **matrix[row][col]** from **cost** and made two recursive calls, one to the cell above and one to the cell on the left. When **row** or **col** reached 0, I followed the only remaining direction, so the code never reads outside the matrix. If **cost** dropped below 0, I returned 0 immediately, which cuts off paths that have already spent too much. Only two branches reached (0, 0) with exactly 4 left, which matches **matrix[0][0]**, so the base case returned 1 for each. These were the paths 4, 7, 1, 3, 1, 6, 3 and 4, 5, 7, 3, 1, 2, 3, which both add up to 25. The recursion added the two 1s together and printed **Number of Paths: 2**.

Screenshots of demonstration (Abello)

Test Case 1 : 25 Target Cost



There are two valid paths for the target cost of 25.



Path 1:
Step 1 : (0,0) = 4
Step 2 : (0,1) = 11
Step 3 : (0,2) = 12
Step 4 : (1,2) = 15
Step 5 : (2,2) = 16
Step 6 : (3,2) = 22
Step 7 : (3,3) = 25

The path that it takes was Right, Right, Down, Down, Down, Right.

Path 2:
Step 1 : (0,0) = 4
Step 2 : (1,0) = 9
Step 3 : (1,1) = 16
Step 4 : (1,2) = 19
Step 5 : (2,2) = 20
Step 6 : (2,3) = 22
Step 7 : (3,3) = 25

The path that it takes was Down, Right, Right, Down, Right, Down.

Since both paths reached the last step with nothing remaining, each one returns 1, and the two are added together to give 2.

Test Case 2 : 22 Target Cost



Since our target cost in case 3 has no valid path, it returned the value 0. There are two ways the code fails here. A path that costs too much, like 4 + 7 + 1 + 6 + 9 + 2 + 3 = 32, makes **cost** go negative (22 - 32 = -10), so **cost** < 0 returns 0. A path that costs too little, like the cheapest path 4 + 5 + 3 + 2 + 1 + 2 + 3 = 20, reaches (0,0) with 6 still remaining, but **matrix[0][0]** is 4, so the base case returns 0. No path in this matrix totals exactly 22, since the costs jump from 21 to 24, so every branch returns 0.

CODE (Domingo)

```cpp
#include <iostream>

int countPaths(int matrix[][4], int row, int col, int cost)
{
    if (cost < 0)
    {
        return 0;
    }

    if (row == 0 && col == 0)
    {
        return (cost == matrix[0][0]) ? 1 : 0;
    }

    if (row == 0)
    {
        return countPaths(
            matrix,
            0,
            col - 1,
            cost - matrix[row][col]
        );
    }

    if (col == 0)
    {
        return countPaths(
            matrix,
            row - 1,
            0,
            cost - matrix[row][col]
        );
    }

    return countPaths(
        matrix,
        row - 1,
        col,
        cost - matrix[row][col]
    )
    +
    countPaths(
        matrix,
```

```
            row,
            col - 1,
            cost - matrix[row][col]
        );
}

int main()
{
    int matrix[4][4] =
    {
        {2, 5, 3, 1},
        {4, 1, 6, 2},
        {7, 3, 2, 5},
        {1, 4, 2, 3}
    };

    int cost = 22;

    std::cout << "Target Cost: "
              << cost << std::endl;

    std::cout << "Number of Paths: "
              << countPaths(matrix, 3, 3, cost)
              << std::endl;

    return 0;
}
```

Screenshots of demonstration (Domingo)

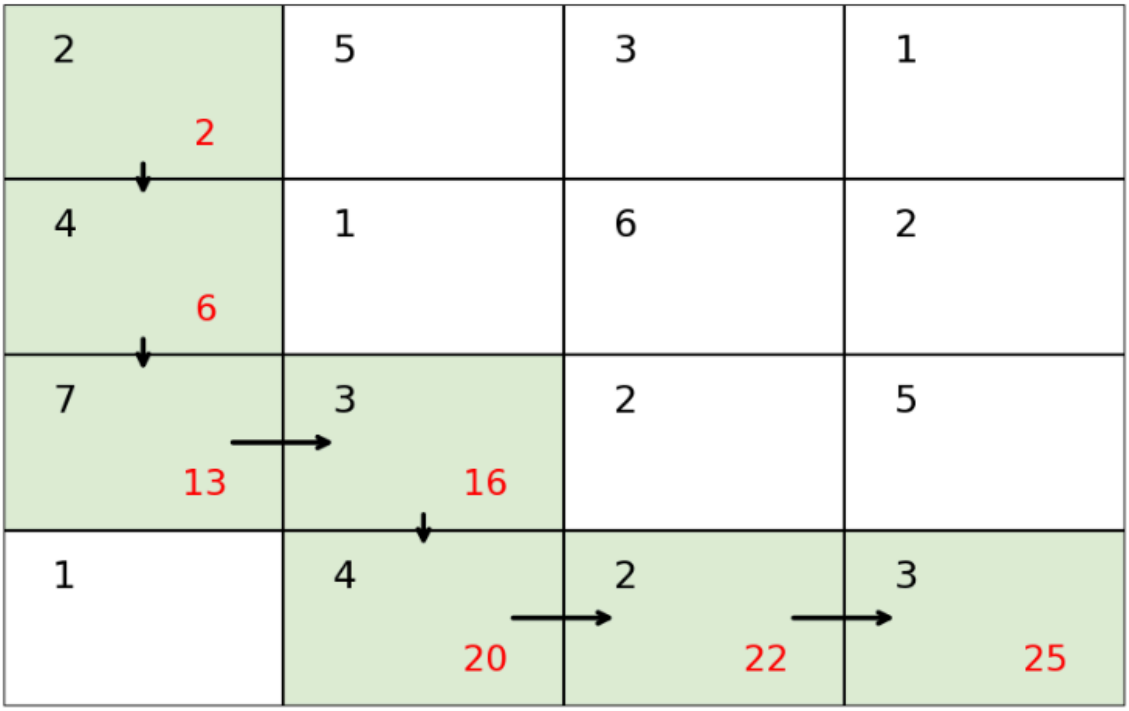
Test Case 1 : 25 Target Cost

```
Target Cost: 25
Number of Paths: 1
```

There is one valid path for the target cost of 25. The diagram is

Target Cost: 25 | Number of valid paths: 1

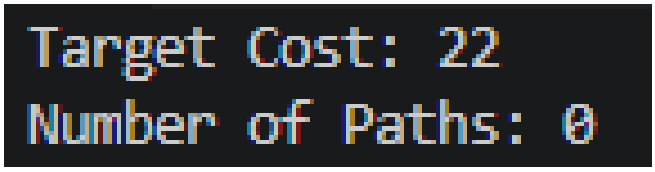
Path 1 (total = 25)



Step 1 : (0,0) = 2
Step 2 : (1,0) = 6
Step 3 : (2,0) = 13
Step 4 : (2,1) = 16
Step 5 : (3,1) = 20
Step 6 : (3,2) = 22
Step 7 : (3,3) = 25

The path that it takes was Down, Down, Right, Down, Right, Right. Since we reached the last step without any remaining cost left, it will return 1.

Test Case 2 : 22 Target Cost



Since our target cost in case 3 has no valid path, it returned the value 0. A path that costs too much, like 2 + 5 + 3 + 1 + 6 + 2 + 5 + 3... is not even needed to see this, because any path that totals 23 or more makes **cost** go negative before the start. A path that costs too little, like 2 + 5 + 1 + 3 + 2 + 2 + 3 = 18, reaches (0,0) with 6 remaining, but **matrix[0][0]** is 2, so the base case returns 0. No path in this matrix totals exactly 22, since the costs jump from 21 to 23, so every branch returns 0.

Analysis of the Working Code (Domingo)

I ran **countPaths(matrix, 3, 3, 22)**, which starts at the last cell and works backward toward (0,0). At each cell, I subtracted **matrix[row][col]** from the remaining cost and made two recursive calls, one to the cell above and one to the cell on the left. When the row or column reached 0, the code followed the only remaining direction, preventing it from accessing cells outside the matrix. If the remaining cost dropped below 0, the function returned 0 to stop exploring that path. Only one path reached (0,0) with exactly 2 remaining, matching the starting cell's value. This path was 3, 2, 5, 2, 6, 1, 3, which adds up to 22. The recursion counted this valid path and printed Number of Paths: 1.

Conclusion + Findings/Observation (Domingo)

Through this activity, I learned about different algorithmic strategies such as recursion, brute force, backtracking, greedy algorithms, divide-and-conquer, and dynamic programming. I understood how each strategy solves problems in different ways and how memoization and bottom-up dynamic programming can improve efficiency by storing previously calculated results. In the supplementary activity, I used recursion to count the number of paths that matched the target cost by checking the cells above and to the left while making sure the cost did not become negative. I think I did well in completing the activity and understanding how the code works, but I still need to improve my skills in tracing recursive calls and analyzing different test cases. Overall, this activity helped me improve my problem-solving skills and understand the importance of choosing the right algorithm for a specific problem.

Conclusion + Findings/Observation (Abello)
In this activity, I learned how different algorithmic strategies work and how they can be applied to solve programming problems. By studying the procedures and analyzing the codes, I gained a better understanding of recursion, searching, sorting, backtracking, and dynamic programming. For the supplementary activity, I analyzed how the recursive function checks possible paths in a matrix and counts only those that match the given target cost. I believe I performed well because I was able to understand the purpose of the conditions and recursive calls, although I still need more practice in debugging code and following complicated recursive processes. This activity was helpful in developing my logical thinking, and I will continue practicing different algorithms to improve my accuracy and programming skills.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT & Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction & summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.